QCon
全球软件开发大会

InfoQ 技术大会

# 当Vibe Coding撞上企业级现实

## 一个软件团队如何重建协作、责任与成长


**李榕**

微软CoreAI 软件开发经理



QCon 上海 2026 · 打造AI Native型组织




# 三个月的兴奋，换来一次集体刹车

第一幕 · 团队突然变快了，为什么反而停了下来？

## 提速

Agentic 工作流跑起来了，功能交付更快，承诺也更多。

## 代价显现

Bug、投诉与返工增加，修复又带来新的问题。

## 集体刹车

全员同意暂停新功能开发，集中修 Bug、恢复稳定性。

停下来是正确的决定，但原本可以不用走到这一步。



QCon 全球软件开发大会
InfoQ 技术大会

# 没有人失职，但组织没有跟上

第一幕 · 团队突然变快了，为什么反而停了下来？

## 当时看得见的

- 管理层看到交付变多了
- 工程师看到代码能跑

Coding 提速立刻可见，两边都没有看错。

## 当时看不见的

- 质量问题可能几个月后才暴露
- 技术债还在不断积累

代价不是不存在，只是还没显现。

我们也没有留下「之前」的基线：先建立度量，再开始提速。



QCon 全球软件开发大会
InfoQ 技术大会

# AI 能生成更多代码，人却没更多精力 Review

第二幕 · 变的不只是工具，还有工作职责

### 逐行 Review

代码越来越多，人却看不过来了。

### 不 Review

眼前交付更快，质量问题留到以后。

### 走过场

看起来仍在把关，实际上问题只是更晚暴露。

能生成更多代码，不等于能安全地交付更多功能。



QCon 全球软件开发大会
InfoQ 技术大会

# 工作已经变了，头衔还没变

第二幕 · 变的不只是工具，还有工作职责

## IC 的工作重心正在转变

理解问题 → 设计方案 → 验证结果 → Sign-off

这里面首先消失的就是coding，这是工程师职责最明显的一次转变。
写代码仍然重要，但判断该做什么、是否做对、能否交付，更关键。

经验的价值，更多体现在判断，而不只是产出速度。



QCon 全球软件开发大会
InfoQ 技术大会

# 把质量与系统健康，变成团队的共同目标

第三幕 · 重新组织团队，而不是要求大家更努力

| | |
|---|---|
| **验证能力** | 像做产品一样建设：有效的测试用例、集成测试和独立人工验证。 |
| **系统级技术债** | 配专门资源，持续重构、删除、重写；不要等「有空再说」。 |
| **系统整体** | 有人对系统整体负责，而不只是各自完成手里的 Issue。 |

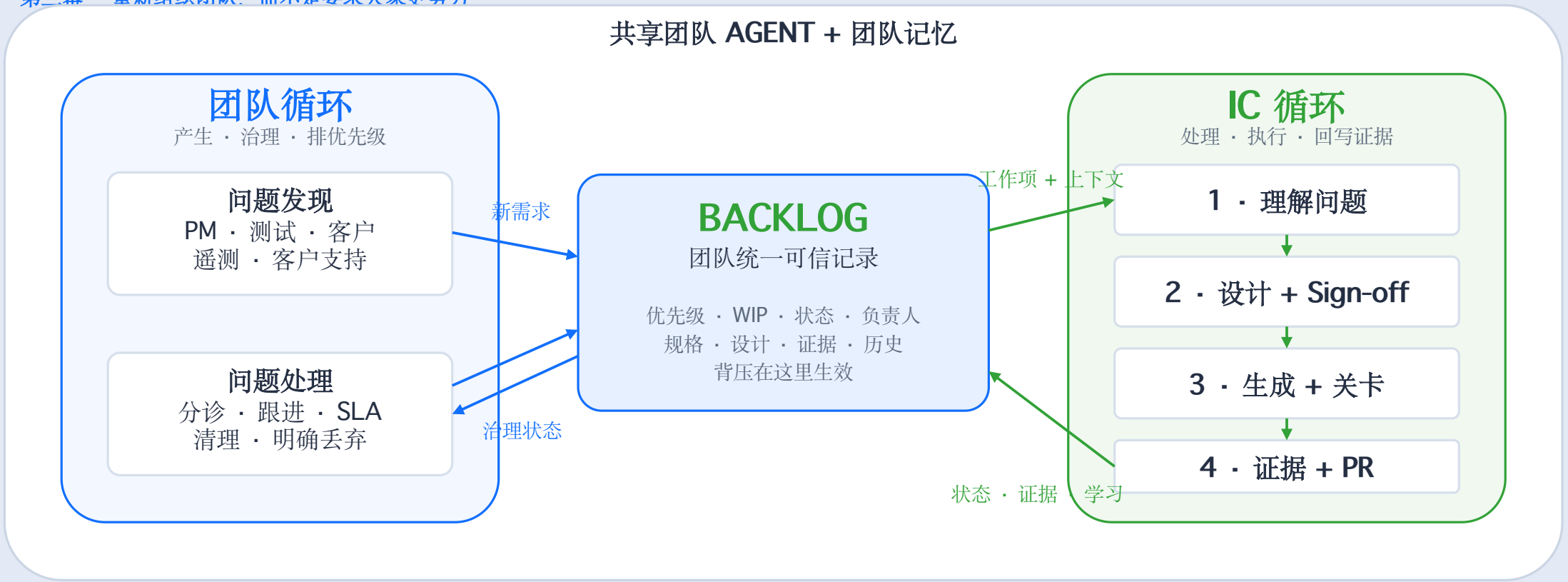
守住质量 → 控制技术债 → 释放速度：这是依赖关系，不是口号。



QCon 全球软件开发大会
InfoQ 技术大会

# Backlog 是团队协作的中心






共享 agent 跨越信息系统边界；上下文不只留在个人脑中和聊天记录里。



QCon 全球软件开发大会
InfoQ 技术大会

# 不能只自动发现问题，却没人处理

第三幕 · 重新组织团队，而不是要求大家更努力

## 发现问题

需求、测试、客户反馈和遥测，持续往 Backlog 添加工作。

## 处理问题

分诊、跟进进展、盯住 SLA，及时关闭过时的 Issue。

## 控制积压

设 WIP 上限，明确哪些不做，把清理 Backlog 当成团队目标。

只自动化发现、不自动化处置，等于批量生产焦虑。



QCon 全球软件开发大会
InfoQ 技术大会

# 一个 Issue，从理解问题到提交 PR

第三幕 · 重新组织团队，而不是要求大家更努力

| | | |
|---|---|---|
| 1 | 理解问题 | Bug 做根因分析；需求做方案探索，需要时找 DRI 讨论。 |
| 2 | 设计与审批 | 写下设计和执行计划，由人在生成代码前批准。 |
| 3 | 生成与检查 | 在同一会话内生成代码、运行检查和测试，并保留结果。 |
| 4 | 提交 PR | 附上设计与测试证据，把进展和结论记录到 Issue 上。 |

把来龙去脉记在 Issue 上，减少反复解释，也让责任有据可查。



QCon 全球软件开发大会
InfoQ 技术大会

# 把「还原意图」交给机器，把判断留给人

第四幕 · 自动化之后，责任更清楚，还是更模糊？

## 人先批准设计

先讨论目标与边界，在生成代码前确定设计。

## 独立 agent 检查

只读 diff，还原实现思路，与已批准的设计对照。

## 人作最终判断

看设计差异和测试证据，决定是否接受这个改动。

符合设计 ≠ 设计正确。检查实现是否符合设计，不能替人判断设计对不对。

# Sign-off：不必逐行读完，但要有依据地负责

第四幕 · 自动化之后，责任更清楚，还是更模糊？

| | |
|---|---|
| **人批准什么** | 问题定义、设计、边界与风险；明确由谁承担变更责任。 |
| **依据是什么** | 设计差异、测试结果和证据；风险越高，Review 越要深入。 |
| **系统如何把关** | 自动检查 → Merge Queue → 人工验证 → 灰度与回滚。 |

每一层都可能漏检；有人批准，不代表其他检查就可以省掉。



QCon 全球软件开发大会
InfoQ 技术大会

# 检查「全绿」以后，别让 Review 变成走过场

第四幕 · 自动化之后，责任更清楚，还是更模糊？

## 通过不等于正确

检查通过了，也不能保证没问题，批准仍然要有依据。

## 不要只给一个绿灯

展示设计差异，而不只是对勾，并随机抽取 PR 做深入 Review。

## 把教训写进检查

漏检后追查哪项检查缺失，能写成明确规则，就不让 AI 判断。

抽查、缺陷演练、审批过快告警：这些办法还需要验证，不是现成的答案。



QCon 全球软件开发大会
InfoQ 技术大会

# 同一台放大器，对不同的人意味着什么



## 有系统经验的人

- 见过设计取舍和边界情况
- 能发现局部没错、整体却有问题
- 更能把握 AI 生成代码的风险

## 经验尚在积累的人

- 还不知道哪些问题值得怀疑
- 一次误解可能被反复复制
- 更难及时判断影响范围

这是我们对经验差异的观察，不是给新人贴上「能力不足」的标签。

# 我们消除的摩擦，也可能是学习机会



**过去：做准备工作 → 犯错 → 收到反馈 → 理解系统**

现在：为了提速，我们把越来越多的基础工作交给 AI。

**省掉的可能不只是重复劳动，也包括理解细节、积累判断力的机会。**



QCon 全球软件开发大会
InfoQ 技术大会

# 下一代资深工程师，从哪里来？

第五幕 · 今天的效率，会不会消耗明天的人才？

判断力依赖经验，
但积累经验的路径正在被我们改写。

如果准备工作都由 AI 完成，
新人如何成长为能作判断、能承担责任的人？

这是我尚未解决的问题。未来如何培养新人，你怎么想？



QCon 全球软件开发大会
InfoQ 技术大会

# 提速省下来的时间，团队决定怎么用

第五幕 · 今天的效率，会不会消耗明天的人才？

## 系统健康

验证体系与技术债，不能等有空再投入。

## 团队记忆

记录设计、决策和验证结果，让团队都能查到。

## 人的成长

主动安排练习与反馈，保留新人的成长路径。

不要把省下来的时间全部用来做更多功能；系统和人，也需要投入。



QCon 全球软件开发大会
InfoQ 技术大会

# 提速之后，要重新分配三样东西

收束 · 团队的能力，不只是代码产量

| | |
|---|---|
| **注意力** | 人专注于理解问题、设计和判断；重复操作与自动检查交给系统。 |
| **责任** | 明确谁批准、依据什么批准；不让全绿替代人的判断。 |
| **学习机会** | 新人怎样接触细节、获得反馈，最终成长为能承担责任的人。 |

持续交付、清醒判断，并培养出下一批能承担责任的人。



QCon 全球软件开发大会
InfoQ 技术大会

QCon
全球软件开发大会

InfoQ 技术大会

# 谢谢大家

欢迎提问与交流

李榕 · 微软 软件开发经理



QCon
全球软件开发大会

InfoQ 技术大会

# 谢谢大家

欢迎提问与交流

李榕 · 微软 软件开发经理



QCon
全球软件开发大会

InfoQ 技术大会

# 谢谢大家

欢迎提问与交流

李榕 · 微软 软件开发经理



QCon
全球软件开发大会

InfoQ 技术大会

# 谢谢大家

欢迎提问与交流

李榕 · 微软 软件开发经理



QCon
全球软件开发大会

InfoQ 技术大会

# 谢谢大家

欢迎提问与交流

李榕 · 微软 软件开发经理

